


# SDEV2171

## Lesson 13: Loading, Errors, and Race Conditions

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

# Lesson Objectives

By the end of this lesson, you should be able to:

- Show loading, success, and error states for a request
- Model a request with one status value instead of separate flags
- Tell an HTTP error from a network error and handle both
- Cancel out-of-date requests with `AbortController`
- Reach a local server from a browser, simulator, emulator, or phone

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Today

| Time | Activity |
|---|---|
| 0:00 | Start the backend, point your app at it |
| 0:10 | What lesson 12’s screen gets wrong |
| 0:20 | Challenge 1: request states |
| 0:45 | Break |
| 0:55 | Responses that arrive out of order |
| 1:00 | Challenge 2: search without stale results |
| 1:20 | Extensions |
| 1:35 | Checkpoint |

WE ARE ESSENTIAL TO ALBERTA | NAIT

# The Backend

```
cd backend
node server.js
```

`GET http://localhost:3000/api/study-steps` returns `[{ "id": "1", "label": "..." }, ...]`

| Query | Effect |
|---|---|
| `q=che` | labels containing "che" |
| `delay=1500` | answer after 1500 ms |
| `error=1` | answer with a 500 and `{ "error": "..." }` |
| `race=1` | shorter searches answer slower |

The terminal logs each request, and every one the app cancels.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# localhost Depends on Where the App Runs

| App running in | Your computer is at |
|---|---|
| Browser | `localhost` |
| iOS simulator | `localhost` |
| Android emulator | `10.0.2.2` (the emulator's own `localhost` is the emulator) |
| Physical phone | your computer's LAN IP, on the same Wi-Fi |

```
const HOST = Platform.OS === 'android' ? '10.0.2.2' : 'localhost';
export const API_BASE_URL = `http://${HOST}:3000/api`;
```

Development builds allow plain `http`. Release builds block it by default on both platforms.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# What Lesson 12’s Screen Shows

| Situation | Screen shows | Should show |
|---|---|---|
| Request still running | "No study steps yet." | a loading indicator |
| Server answered 500 | "No study steps yet." | the error, and a way to retry |
| Server unreachable | "No study steps yet." | the error, and a way to retry |
| Search matched nothing | "No study steps yet." | correct |

One message for four situations. The user can’t tell a slow network from a broken app.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Flags or One Status

```
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);
const [data, setData] = useState([]);
// isLoading && error is possible. What does the screen show?
```

```
const [status, setStatus] = useState('loading'); // 'loading' | 'success' | 'error'
const [steps, setSteps] = useState([]);
const [errorMessage, setErrorMessage] = useState('');
```

With one `status`, the screen is always in exactly one state. Each path through the request sets it once, so there's no flag left to reset in `finally`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Two Kinds of Failure

| | HTTP error | Network error |
|---|---|---|
| Example | 404, 500 | server down, wrong host, offline |
| `fetch` | resolves | rejects with a `TypeError` |
| You detect it by | checking `response.ok` | the `catch` |
| Message | the response body, if the API sends one | `Failed to fetch` (browser), `Network request failed` (device) |

```
if (!response.ok) {
  const body = await response.json().catch(() => ({}));
  throw new Error(body.error ?? `Server returned ${response.status}`);
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Challenge 1: Request States

Replace `remoteSteps` with `status`, `steps`, and `errorMessage`. Add `?${TEST_PARAMS}` to the URL.

1. Loading: `ActivityIndicator` and "Loading study steps…"
2. Success: the list
3. Server error: the message from the response body
4. Any error: a Retry button that runs the request again

Test with `'delay=1500'`, then `'delay=1500&error=1'`, then with the server stopped. Then restart it and Retry.

Full spec and solution: README, Challenge 1.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Break

10 minutes

WE ARE ESSENTIAL TO ALBERTA
NAIT

# Responses Arrive Out of Order

With `race=1`, type `che` with 150 ms between keys:

| Request | Sent at | Answers at |
|---|---|---|
| `q=c` | 0 ms | 1600 ms |
| `q=ch` | 150 ms | 1350 ms |
| `q=che` | 300 ms | 1100 ms |

The last response to arrive is for `c`. The screen shows 9 results while the box says `che`.

Real networks reorder responses too. `race=1` just makes it happen every time.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Challenge 2: Search Without Stale Results

1. Add `query` state, a `TextInput`, and `q=${encodeURIComponent(query)}` in the URL
2. Move the request into `useEffect(..., [query, reloadKey])`. Retry does `setReloadKey((key) => key + 1)`
3. Set `TEST_PARAMS = 'race=1'`. Type `che` quickly and get 9 results
4. Fix it

Done when `che` shows 3 results and the server logs `cancelled by the app`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Cancel the Old Request in Cleanup

```
useEffect(() => {
  const controller = new AbortController();
  async function loadSteps() {
    try {
      const response = await fetch(url, { signal: controller.signal });
      // ...
    } catch (error) {
      if (controller.signal.aborted) return;
      // ...
    }
  }
  loadSteps();
  return () => controller.abort();
}, [query, reloadKey]);
```

React runs the cleanup before the effect runs again, and when the screen unmounts. An aborted `fetch` rejects, so ignore it in `catch`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Extensions

| Extension | Done when |
| --- | --- |
| Debounce: wait 300 ms after the last keystroke | typing `che` sends one request |
| Timeout: give up after 5 s | `delay=8000` shows a timeout message |
| Server down: a readable network error | stopped server shows "Can’t reach the server. Is it running?" |
| Custom hook: `useStudySteps(query, reloadKey)` | the screen gets `{ status, steps, errorMessage }` from it |

WE ARE ESSENTIAL TO ALBERTA | NAIT

# What Production Apps Use

Libraries such as TanStack Query and SWR do all of today's work for you:

- `status`, data, and error for each request
- cancelling out-of-date requests
- retries, caching, and refetching data that may be out of date

Today you built the core of what they do. Knowing it makes their options and bugs much easier to understand.

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Checkpoint

- What does your screen show for a 500, and for a stopped server? Why does the code reach them differently?
- What state does `isLoading` plus `error` allow that `status` doesn't?
- In the race, which request answered last, and why did it win?
- When does an effect's cleanup run?

WE ARE ESSENTIAL TO ALBERTA | NAIT

# Before You Leave

1. Your screen handles loading, success, server error, network error, and stale searches
2. Set `TEST_PARAMS` back to `'delay=1500'` or remove it
3. Record your status: **resolved**, **partial**, or **blocked**
4. Exercise: see the end of the lesson README

WE ARE ESSENTIAL TO ALBERTA | NAIT